

Parallel Ranges

J. Daniel García

Universidad Carlos III de Madrid



Creative Commons License

© This work is distributed under the **Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)** license.

You are free to:

Share – copy and redistribute the material in any medium or format.

Adapt – remix, transform, and build upon the material.

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

- ① **Attribution** – You must give **appropriate credit**, provide a link to the license, and **indicate if changes were made**. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- ⑤ **NonCommercial** – You may not use the material for **commercial purposes**.
- ③ **ShareAlike** – If you **remix, transform, or build upon** the material, you must distribute the modified material under the same license.

No additional restrictions – You may not apply legal terms or **technological measures** that legally restrict others from doing anything the license permits.

Complete text of the license available at

<https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.en>

1 Ranges

2 Parallel algorithms with ranges

1 Ranges

- Ranges
- Views
- Example: Counting words in a text

- A range is any type that represents a sequence of elements.

```
template <typename T>  
concept range = requires(T& t) {  
    { std::ranges::begin(t) };  
    { std::ranges::end(t) };  
};
```

- `[std::ranges::begin(t), std::ranges::end(t))` is a half-open range.
- `std::ranges::begin(t)` and `std::ranges::end(t)` must be valid expressions; their complexity depends on the range type.
- If the type of `std::ranges::begin(t)` is a forward iterator, then `[std::ranges::begin(t), std::ranges::end(t))` is equality preserving (i.e., multi-pass is allowed).

A ranges taxonomy

- **range**: Can be iterated from begin to end.
- **output_range**: Elements can be written to the range.
- **input_range**: Elements can be read from the range.
- **forward_range**: Elements can be read from the range multiple times.
- **bidirectional_range**: Elements can be read from the range in both directions.
- **random_access_range**: Elements can be read from the range in constant time.
- **contiguous_range**: All elements are stored contiguously in memory.
- **sized_range**: Range with constant-time size.

Range based algorithms

- Ranges are the basis for a new set of algorithms in the STL.
- These algorithms take ranges as input and output, instead of pairs of iterators.
- The new algorithms are designed to be more composable and easier to use than the old ones.

Range based algorithms

- Ranges are the basis for a new set of algorithms in the STL.
- These algorithms take ranges as input and output, instead of pairs of iterators.
- The new algorithms are designed to be more composable and easier to use than the old ones.

```
void f() {  
    std::vector<double> v = get_vector();  
    std::ranges::transform(v, v.begin(), [](double x) { return x * 2; });  
    std::ranges::reverse(v);  
    if (auto it = std::ranges::find_if(v, [](double x) { return x > 0.0; }); it != v.end()) {  
        std::println("Found a positive number: {}", *it);  
    }  
}
```


1 Ranges

- Ranges
- Views
- Example: Counting words in a text

- A view is a lightweight range type, typically lazy and often non-owning.
- Range adaptors such as `std::views::take` are used to create views from existing ranges, without copying the elements.
- Views are typically lazy, meaning that they do not compute their elements until they are accessed.

- A view is a lightweight range type, typically lazy and often non-owning.
- Range adaptors such as `std::views::take` are used to create views from existing ranges, without copying the elements.
- Views are typically lazy, meaning that they do not compute their elements until they are accessed.

```
void f() {  
    std::vector<double> v = get_vector();  
  
    // Create a view and traverse it  
    auto v1 = std::views::take(v, 10);  
    for (auto const & x : v1) {  
        std::println("{} ", x);  
    }  
  
    // Traverse a temporary view  
    for (auto const & x : std::views::take(v, 10)) {  
        std::println("{} ", x);  
    }  
}
```

Chaining views

- Views can be chained together to create more complex ranges.
- The resulting range is still a view, and is still lazy.

Chaining views

- Views can be chained together to create more complex ranges.
- The resulting range is still a view, and is still lazy.

```
void f() {  
    std::vector<double> v = get_vector();  
    auto v1 = std::views::filter(v, [](double x) { return x > 0.0; });  
    auto v2 = std::views::transform(v1, [](double x) { return x * 2; });  
    auto v3 = std::views::take(v2, 10);  
    for (auto const & x : v3) {  
        std::println("{} ", x);  
    }  
}
```

Chaining and temporary views

- Views can be chained together even if some of them are temporary.
- The resulting range is still a view, and is still lazy.

Chaining and temporary views

- Views can be chained together even if some of them are temporary.
- The resulting range is still a view, and is still lazy.

```
void f() {  
    std::vector<double> v = get_vector();  
    auto v3 = std::views::take(  
        std::views::transform(  
            std::views::filter(v,  
                [](double x) { return x > 0.0; }),  
            [](double x) { return x * 2; }),  
        10);  
    for (auto const & x : v3) {  
        std::println("{} ", x);  
    }  
}
```

Piping views

- Views can also be chained together using the pipe operator.
- The resulting range is still a view, and is still lazy.

- Views can also be chained together using the pipe operator.
- The resulting range is still a view, and is still lazy.

```
void f() {  
    std::vector<double> v = get_vector();  
    auto v3 = v | std::views::filter([](double x) { return x > 0.0; })  
               | std::views::transform([](double x) { return x * 2; })  
               | std::views::take(10);  
    for (auto const & x : v3) {  
        std::println("{} ", x);  
    }  
}
```

1 Ranges

- Ranges
- Views
- Example: Counting words in a text

Example: Counting words

- Counting words in a text:
 - An empty text has 0 words.
 - A new word starts when a letter character follows a non-letter character.
 - If text starts with a letter character, it starts with a word.

Checking for a new word

```
size_t is_letter(char c) {  
    return static_cast<std::size_t>(  
        std::isalpha(static_cast<int>(static_cast<unsigned char>(c))) != 0);  
}  
  
size_t is_new_word(char c1, char c2) {  
    return static_cast<std::size_t>((is_letter(c1) == 0) and (is_letter(c2) != 0));  
}
```

Counting words: Naive approach

- An explicit loop checking consecutive characters.
- The first character is a special case.

Explicit loop

```
size_t count_classic(std::string_view text) {  
    if (text.empty()) { return 0; }  
    size_t result = 0;  
    for (size_t i = 1; i < text.size(); ++i) {  
        if (is_new_word(text[i - 1], text[i]) != 0) { ++result; }  
    }  
    return result + is_letter(text[0]);  
}
```

Using ranges and views

■ Steps:

- Generate a view with the $n - 1$ first characters.
- Generate a view with the $n - 1$ last characters.
- Zip the two views to get a view of pairs of characters.
- Count the number of pairs that start a new word.

Using ranges and views

```
size_t count_ranges(std::string_view text) {  
    if (text.empty()) { return 0; }  
    auto compute = std::views::zip(  
        text | std::views::take(std::ranges::size(text) - 1),  
        text | std::views::drop(1));  
    auto result = std::ranges::count_if(compute, [](auto char_pair) {  
        auto [c1, c2] = char_pair;  
        return is_new_word(c1, c2);  
    });  
    return result + is_letter(text[0]);  
}
```

2 Parallel algorithms with ranges

- Introduction
- Differences with existing algorithms
- Example: Counting words

- Finding with a predicate.

```
std::vector<double> v = get_values();  
auto it = std::find_if(v.begin(), v.end(), [](double x) { return x>0; });
```

- Finding with a predicate.

```
std::vector<double> v = get_values();  
auto it = std::find_if(v.begin(), v.end(), [](double x) { return x>0; });
```

- Finding in parallel with an execution policy and iterators (C++17)

```
std::vector<double> v = get_values();  
auto it = std::find_if(std::execution::par, v.begin(), v.end(), [](double x) { return x>0; });
```


- Finding with a predicate.

```
std::vector<double> v = get_values();  
auto it = std::find_if(v.begin(), v.end(), [](double x) { return x>0; });
```

- Finding in parallel with an execution policy and iterators (C++17)

```
std::vector<double> v = get_values();  
auto it = std::find_if(std::execution::par, v.begin(), v.end(), [](double x) { return x>0; });
```

- Finding in parallel with an execution policy and ranges (C++26).

```
std::vector<double> v = get_values();  
auto it = std::ranges::find_if(std::execution::par, v, [](double x) { return x>0; });
```

- Finding the last element whose square is greater than 100 (C++17).

```
std::vector<double> v = get_values();  
std::transform(std::execution::par, v.begin(), v.end(), v.begin(), [](double x) { return x*x; });  
std::reverse(std::execution::par, v.begin(), v.end());  
auto it = std::find_if(std::execution::par, v.begin(), v.end(), [](double x) { return x>100; });
```

- Finding the last element whose square is greater than 100 (C++17).

```
std::vector<double> v = get_values();  
std::transform(std::execution::par, v.begin(), v.end(), v.begin(), [](double x) { return x*x; });  
std::reverse(std::execution::par, v.begin(), v.end());  
auto it = std::find_if(std::execution::par, v.begin(), v.end(), [](double x) { return x>100; });
```

- **Drawbacks:**

- Three separate algorithms, each with its own overhead.
- Temporary intermediate computations are performed eagerly, so they cannot be skipped even if a later search could short-circuit.

- Finding the last element whose square is greater than 100 (C++17).

```
std::vector<double> v = get_values();  
std::transform(std::execution::par, v.begin(), v.end(), v.begin(), [](double x) { return x*x; });  
std::reverse(std::execution::par, v.begin(), v.end());  
auto it = std::find_if(std::execution::par, v.begin(), v.end(), [](double x) { return x>100; });
```

- **Drawbacks:**

- Three separate algorithms, each with its own overhead.
- Temporary intermediate computations are performed eagerly, so they cannot be skipped even if a later search could short-circuit.

- Combining ranges and parallel execution policies (C++20).

```
std::vector<double> v = get_values();  
auto compute = v  
    | std::views::transform([](double x) { return x*x; })  
    | std::views::reverse;  
auto it = std::find_if(std::execution::par,  
    std::begin(compute), std::end(compute),  
    [](double x) { return x>100; });
```

Combining algorithms with ranges

- Combining ranges and parallel execution policies (C++26).

```
std::vector<double> v = get_values();  
auto compute = v  
    | std::views::transform([](double x) { return x*x; })  
    | std::views::reverse;  
auto it = std::ranges::find_if(std::execution::par,  
    compute,  
    [](double x) { return x>100; });
```

- **Warning:** The temporary variable **compute** is necessary to avoid a dangling reference.

```
std::vector<double> v = get_values();  
auto it = std::ranges::find_if(std::execution::par,  
    v  
    | std::views::transform([](double x) { return x*x; })  
    | std::views::reverse,  
    [](double x) { return x>100; });
```

2 Parallel algorithms with ranges

- Introduction
- Differences with existing algorithms
- Example: Counting words

Execution policy parameter

- The execution policy parameter can be used with:
 - Iterator based algorithms.
 - Range based algorithms.

- The execution policy parameter can be used with:
 - Iterator based algorithms.
 - Range based algorithms.
- New iterator based parallel algorithms with updated interfaces.

```
ranges::unary_transform_result<I, O>  
ranges::transform(Ep&& exec, I first, S last, O result, OutS result_last,  
                  F op, Proj proj = {});
```

- **Ep**: Execution policy.
- **I, S**: Input iterator and sentinel.
- **O, OutS**: Output iterator and sentinel.
- **F**: Unary operation.
- **Proj**: Projection (optional).

- New range based parallel algorithms with updated interfaces.

```
ranges::unary_transform_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>  
ranges::transform(Ep&& exec, R&& r, OutR&& result_r, F op, Proj proj = {});
```

- **Ep**: Execution policy.
- **R**: Input range.
- **OutR**: Output range.
- **F**: Unary operation.
- **Proj**: Projection (optional).

Parallel algorithms require random access

- Iterator based parallel algorithms require random access iterators.
 - Both input and output iterators must be random access.
- Range based parallel algorithms require random access ranges.
 - Both input and output ranges must be random access.

Parallel algorithms require random access

- Iterator based parallel algorithms require random access iterators.
 - Both input and output iterators must be random access.
- Range based parallel algorithms require random access ranges.
 - Both input and output ranges must be random access.
- **Consequences:**
 - Cannot be used with input, forward, or bidirectional iterators/ranges.
 - Cannot be used with output iterators/ranges that are not random access.
 - Cannot be used with some views (e.g., `std::views::filter`).

Parallel algorithms require random access

- Iterator based parallel algorithms require random access iterators.
 - Both input and output iterators must be random access.
- Range based parallel algorithms require random access ranges.
 - Both input and output ranges must be random access.
- **Consequences:**
 - Cannot be used with input, forward, or bidirectional iterators/ranges.
 - Cannot be used with output iterators/ranges that are not random access.
 - Cannot be used with some views (e.g., `std::views::filter`).
- **Why?:**
 - Fast division of work into chunks.
 - Random access allows for efficient partitioning and load balancing.

Parallel algorithms require bounded ranges

- Iterator based parallel algorithms require sized sentinels.
 - The distance between the input iterator and sentinel must be computable in constant time.
 - The distance between the output iterator and sentinel must be computable in constant time.
- Range based parallel algorithms require sized ranges.
 - The size of the input range must be computable in constant time.
 - The size of the output range must be computable in constant time.

Parallel algorithms require bounded ranges

- Iterator based parallel algorithms require sized sentinels.
 - The distance between the input iterator and sentinel must be computable in constant time.
 - The distance between the output iterator and sentinel must be computable in constant time.
- Range based parallel algorithms require sized ranges.
 - The size of the input range must be computable in constant time.
 - The size of the output range must be computable in constant time.
- **Consequences:**
 - Cannot be used with ranges that do not have a known size.
 - Cannot be used with some views (e.g., `std::views::filter`).
 - When different sizes are involved processing stops when the end of the shorter range is reached.

Parallel algorithms require bounded ranges

- Iterator based parallel algorithms require sized sentinels.
 - The distance between the input iterator and sentinel must be computable in constant time.
 - The distance between the output iterator and sentinel must be computable in constant time.
- Range based parallel algorithms require sized ranges.
 - The size of the input range must be computable in constant time.
 - The size of the output range must be computable in constant time.
- **Consequences:**
 - Cannot be used with ranges that do not have a known size.
 - Cannot be used with some views (e.g., `std::views::filter`).
 - When different sizes are involved processing stops when the end of the shorter range is reached.
- **Why?:**
 - Efficiently divide work into chunks.
 - Fixes safety problems with out of range access.

Fixing a pending issue: output ranges

- Output is specified by a pair of iterators or a range.

```
ranges::unary_transform_result<I, O>  
ranges::transform(Ep&& exec, I first, S last, O result, OutS result_last,  
                  F op, Proj proj = {});
```

```
ranges::unary_transform_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>  
ranges::transform(Ep&& exec, R&& r, OutR&& result_r, F op, Proj proj = {});
```


2 Parallel algorithms with ranges

- Introduction
- Differences with existing algorithms
- Example: Counting words

Counting words: problem and helper functions

- Goal: count words in a text.
- A new word starts when a letter is preceded by a non-letter.
- The first character must be handled separately.

Helper functions

```
bool is_letter(char c) {  
    return std::isalpha(static_cast<unsigned char>(c));  
}  
  
size_t is_new_word(char c1, char c2) {  
    return !is_letter(c1) && is_letter(c2);  
}
```

Using an algorithm: the map-reduce approach

- Map: check for new words in pairs of characters.
 - Sequences: $(t_0, \dots, t_{n-2})$ and $(t_1, \dots, t_{n-1})$.
 - Mapping: `is_new_word(c1,c2)` applied to pairs of characters.
- Reduce: `std::plus` for adding results of the mapping.

Map-reduce approach

```
size_t count_classic_stl(std::string_view text) {  
    if (text.empty()) { return 0; }  
    auto result =  
        std::transform_reduce(std::begin(text), std::prev(std::end(text)),  
                              std::next(std::begin(text)),  
                              0ZU,  
                              std::plus{},  
                              is_new_word);  
    return result + is_letter(text[0]);  
}
```

Using a parallel map-reduce with C++17

- The same map-reduce approach, but with a parallel execution policy.

Parallel map-reduce approach

```
size_t count_parallel(std::string_view text) {  
    if (text.empty()) { return 0; }  
    auto result = std::transform_reduce(std::execution::par,  
        std::begin(text), std::prev(std::end(text)),  
        std::next(std::begin(text)),  
        0,   
        std::plus{},  
        is_new_word);  
    return result + is_letter(text[0]);  
}
```

Using ranges and views

- Steps:
 - Generate a view with the $n - 1$ first characters.
 - Generate a view with the $n - 1$ last characters.
 - Zip the two views to get a view of pairs of characters.
 - Count the number of pairs that start a new word.

Using ranges and views

```
size_t count_ranges(std::string_view text) {  
    if (text.empty()) { return 0; }  
    auto compute = std::ranges::views::zip(  
        text | std::ranges::views::take(std::ranges::size(text) - 1),  
        text | std::ranges::views::drop(1));  
    auto result = std::ranges::count_if(compute, [](auto char_pair) {  
        auto [c1, c2] = char_pair;  
        return is_new_word(c1, c2);  
    });  
    return result + is_letter(text[0]);  
}
```

Using parallel ranges and views with C++26

- The same approach as before, but with a parallel execution policy.
- **Note:** The parallel execution policy is not yet supported by all compilers.
 - Experimental preview in OneAPI DPL.

Using parallel ranges and views

```
size_t count_pranges(std::string_view text) {  
    namespace stdx = oneapi::dpl;  
  
    if (text.empty()) { return 0; }  
    auto compute =  
        std::ranges::views::zip(text | std::ranges::views::take(std::ranges::size(text) - 1),  
                                text | std::ranges::views::drop(1));  
    auto result = stdx::ranges::count_if(stdx::execution::par_unseq, compute, [](auto char_pair) {  
        auto [c1, c2] = char_pair;  
        return is_new_word(c1, c2);  
    });  
    return result + is_letter(text[0]);  
}
```

- Counting the words in “**Don Quixote**” by Miguel de Cervantes.
- **Number of characters:** 2,225,806
- **Number of words:** 413,338

- Counting the words in “**Don Quixote**” by Miguel de Cervantes.
- **Number of characters**: 2,225,806
- **Number of words**: 413,338

Implementation	Average time (ms)	Speedup
Classic	13.756	1.00
Classic STL	8.004	1.72
Parallel STL	1.824	7.54
Ranges	12.789	1.08
Parallel Ranges	1.179	11.67

Parallel Ranges

J. Daniel García

Universidad Carlos III de Madrid

